ТЕХНИЧЕСКОЕ ЗАДАНИЕ

на разработку ИИ-ассистента

Название проекта:	**Умный Telegram-бот отдела продаж стройматериалов с автоматическим расчетом цен и логистики**
Дата:	26.09.2026
Версия:	2.0 (As-Built / Фактическая реализация)
Автор:	Филимонов Тимофей Юрьевич

1. Общая информация о проекте

1.1 Описание компании/контекста

Сфера деятельности — Торгово-дистрибьюторская компания. Продажа оборудования и комплектующих (около 500 уникальных SKU). Клиентская база — Смешанная: розничные покупатели (физлица) и оптовые B2B-клиенты.

Текущие процессы: Запросы на цены, наличие и доставку поступают через Telegram. Менеджеры обрабатывают их вручную: проверяют прайс, рассчитывают логистику.

Инфраструктура: Проект развернут на двух серверах (РФ и Нидерланды) для обеспечения отказоустойчивости и корректной маршрутизации вебхуков Telegram.

1.2 Проблема, которую решаем

Менеджеры тратят до 40% рабочего времени на рутинные операции: проверку цен, уточнение остатков и ручной расчет доставки. Это приводит к задержкам ответов (до 12 часов) и потере «горячих» лидов.

2. Цели и задачи

2.1 Главная цель проекта

Создать Telegram-бота, который будет автоматически обрабатывать запросы клиентов по 500 SKU, автоматически определять тип клиента (розница/опт) по Telegram ID, консультировать по товарам, рассчитывать стоимость доставки на основе жестких SQL-правил и формировать текстовые ответы — чтобы сократить время ответа до 15 секунд.

2.2 Измеримые показатели успеха (KPI)

Метрика	Целевое значение	Как измеряется
Скорость первичного ответа	До 15 секунд	Время между сообщением и ответом бота
Точность распознавания товара	≥ 90%	Корректный поиск через FTS/Trigram в БД
Точность расчетов	100%	Цены и доставка берутся из SQL, LLM не считает
Полнота логирования	100% диалогов	Запись в таблицу `dialogue_logs` с учетом токенов

3. Пользователи и сценарии

3.1 Целевые пользователи

Группа 1: Розничные клиенты. Физлица. Пишут в Telegram. Ожидают мгновенный ответ с розничной ценой.

Группа 2: Оптовые клиенты. Зарегистрированные дистрибьюторы. Их Telegram `chat_id` уже внесен в базу данных. Ожидают оптовую цену и скидку.

3.2 Пользовательские сценарии (Use Cases)

Сценарий 1. Быстрая проверка цены и наличия

- Триггер: Сообщение: «Сколько стоит перфоратор Bosch и сколько их на складе?»
- Действие бота: 1. Определяет `chat_id`. 2. Ищет товар в БД через полнотекстовый поиск. 3. Подставляет цену (розница/опт) в промпт.
- Результат: Ответ за 10 сек с точной ценой и остатком.

Сценарий 2. Расчет заказа с доставкой

- Триггер: Сообщение: «Нужно 50 болгарок Makita с доставкой в Бишкек».
- Действие бота: 1. Находит товар. 2. Через SQL-джойн находит зону доставки для города «Бишкек» и ее стоимость. 3. Передает все цифры в GigaChat.
- Результат: Сформированный текст с итоговой суммой и сроками.

4. Функциональные требования

4.1 Основные функции (Must Have)

1. Приём запросов: Только через Telegram Bot API.
2. Автоматическая идентификация клиента: Проверка `chat_id` в таблице `clients`. Если клиент найден — применяется его тип (опт/розница) и процент скидки.
3. Умный поиск товаров: Использование PostgreSQL Full-Text Search (FTS) и Trigram similarity для распознавания названия товара из свободного текста пользователя.
4. Расчет доставки через БД: Поиск города в таблице синонимов (`city_synonyms`) и джойн с таблицей правил доставки (`delivery_rules`) для получения точной стоимости и сроков.
5. Генерация ответа через LLM: Формирование финального текста через GigaChat на основе *строго переданных* цифр из БД.
6. Логирование: Запись каждого диалога в таблицу `dialogue_logs` (текст клиента, ответ бота, использованные токены, тип клиента).

4.2 Желательные функции (Should Have)

1. Анализ «слепых зон» через выгрузку из `dialogue_logs`.
2. Различение тона ответа (краткий для Telegram).

4.3 Исключения (Won't Have)

1. Канал Email (не реализован).
2. Сложная авторизация по ИНН/Email в реальном времени (не реализована, используется только `chat_id`).
3. Векторный поиск (RAG/pgvector) для регламентов (не реализован, доставка ищется через SQL).
4. Эскалация на менеджера отдельной нодой (не реализована, fallback обрабатывается самим LLM).
5. Генерация PDF, интеграция с 1C/CRM, голосовые сообщения.

5. Данные и база знаний

5.1 Источники данных

Источник	Формат	Метод поиска
Прайс-лист и остатки	PostgreSQL (`products`)	FTS (`tsvector`) + Trigram (`pg_trgm`)
База клиентов и скидки	PostgreSQL (`clients`)	Точное совпадение по `chat_id`

Правила доставки и города	PostgreSQL (`delivery_rules`, `city_synonyms`)	SQL JOIN по названию города

5.2 Требования к данным

1. Конфиденциальность: Оптовые цены выдаются только если `chat_id` клиента есть в таблице `clients` с типом `wholesale`.
2. Целостность: LLM запрещено вычислять математику. В промпт передаются только готовые числа из SQL-запросов.

6. Технические требования

6.1 Платформа и интеграции

Компонент	**Выбранный инструмент**	**Обоснование / Фактическая реализация**
Интерфейс	Telegram Bot API	Единственный канал связи. Быстрый запуск через Webhook.
LLM провайдер	GigaChat-3-Ultra	Российская модель, 152-ФЗ. В коде жестко задана модель `GigaChat-3-Ultra`.
Оркестратор	n8n (self-hosted)	Визуальное построение пайплайна. Развернут на отдельном сервере.
Хранилище	PostgreSQL	Единая БД. Используются расширения `pg_trgm` (для нечеткого поиска товаров) и стандартный FTS. Векторная БД (pgvector) не используется.

Инфраструктура	2 сервера (РФ + Нидерланды)	Распределенная архитектура. Сервер в РФ хранит БД и обеспечивает合规 (compliance) для GigaChat. Сервер в Нидерландах обрабатывает вебхуки Telegram и n8n для стабильности.

6.2 Нефункциональные требования

- Время ответа: до 15 секунд.
- Безопасность: API-ключи GigaChat и Telegram хранятся в credentials n8n. Данные клиентов в РФ.

7. Ограничения и риски

7.1 Ограничения проекта

- Бюджет: до 3 000 ₽/мес на API GigaChat.
- Сроки: 2 недели до запуска MVP.

7.2 Риски и митигация

Риск	Митигация (реализовано в коде)
Опечатки в названии товара	Использование Trigram similarity (`similarity > 0.3`) в SQL-запросе.
Галлюцинации LLM в цифрах	Математика исключена. В промпт вставляются только жесткие значения из SQL (`price`, `base_cost`).
Нераспознанный город	SQL возвращает NULL, LLM по системному промпту отвечает: «Регион не найден, уточните город».

8. План реализации

Этап	Задачи	Результат

1. Подготовка БД	Создание таблиц `products`, `clients`, `delivery_rules`, `city_synonyms`. Настройка индексов FTS и Trigram.	Готовая PostgreSQL БД
2. Сборка n8n	Настройка Telegram Trigger, SQL-нод, HTTP Request к GigaChat OAuth и Chat API.	Рабочий пайплайн
3. Промпт-инжиниринг	Написание системного промпта, который запрещает считать и требует использовать только переданные JSON-данные.	Стабильные ответы
4. Деплой	Настройка вебхуков на сервере в Нидерландах, подключение к БД в РФ.	Релиз MVP

9. Критерии приемки

☑ Бот корректно находит товар даже при опечатках (благодаря Trigram).

Оптовая цена показывается только тем `chat_id`, которые есть в таблице `clients`.

☑ Стоимость доставки рассчитывается строго по таблице `delivery_rules`, LLM не выдумывает тарифы.

☑ Все диалоги пишутся в таблицу `dialogue_logs` с подсчетом `tokens_used`.

Система работает стабильно на двух серверах (РФ + Нидерланды).

БЛОК 1. СИСТЕМНЫЙ ПРОМПТ ДЛЯ GigaChat (Фактический)

Ты — ИИ-ассистент отдела продаж компании стройматериалов.

ЖЁСТКИЕ ПРАВИЛА (152-ФЗ и точность данных):

1. ТЫ НЕ СЧИТАЕШЬ МАТЕМАТИКУ. Используй ТОЛЬКО предоставленные ниже цифры из БД.

2. НИКОГДА не выдумывай цены, наличие или условия доставки. Если в данных выше указано "Товар не распознан" или "Регион не найден" — честно сообщи об этом клиенту.

3. НИКОГДА не повторяй ИНН, паспорта, email и телефоны.

4. Если клиент НЕ авторизован (тип: retail), а просит опт, ответь: "Оптовые цены доступны только авторизованным дистрибьюторам."

ДАННЫЕ ИЗ БД (Используй ТОЛЬКО их):

- Клиент: {firstName} (Тип: {clientType}, Скидка: {discount}%)

- Товар: {productContext}

- Доставка: {deliveryContext}

ВОПРОС КЛИЕНТА: "{userText}"

ТВОЯ ЗАДАЧА:

Сформируй краткий, вежливый и деловой ответ (2-4 предложения). Назови точную цену, остаток, стоимость и сроки доставки, если они есть в данных выше. В конце предложи оформить заказ.

БЛОК 2. АРХИТЕКТУРА ДАННЫХ В POSTGRESQL (Фактическая)

```
-- Таблица клиентов (идентификация по chat_id)
CREATE TABLE clients (
    chat_id BIGINT PRIMARY KEY,
    client_type VARCHAR(20), -- 'retail' или 'wholesale'
    discount_percent INT,
    first_name VARCHAR(100)
-- Таблица товаров (с поддержкой нечеткого поиска)
CREATE TABLE products (
    id SERIAL PRIMARY KEY,
    name VARCHAR(255),
    price_retail DECIMAL,
    price_wholesale DECIMAL,
    stock_quantity INT,
    unit VARCHAR(20)
-- Индексы для FTS и Trigram создаются отдельно
-- Справочник синонимов городов и правил доставки
CREATE TABLE city_synonyms (city_name VARCHAR(100), synonym VARCHAR(100));
CREATE TABLE delivery_rules (zone VARCHAR(100), base_cost DECIMAL, delivery_days VARCHAR(50), rule_text TEXT);
-- Логи диалогов
CREATE TABLE dialogue_logs (
    id SERIAL PRIMARY KEY,
    chat_id BIGINT,
    client_type VARCHAR(20),
    user_message TEXT,
    bot_response TEXT,
```

```
tokens_used INT,
channel VARCHAR(20),
created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
```